Northeastern University

Class Assignment 2: EECE 7205

Professor: Dr. Fatema Nafa

Rohini Bidnurmath
NUID: 003916201

October 2026

Contents

1 Problem 1: Creating the Tree **3**

2 Problem 2: Traversing the Tree **5**

3 Problem 3: Adding Elements to the Tree **7**

4 Problem 4: Deleting Elements (Every Case) **9**

5 Problem 5: Deleting All Elements Sequentially **13**

6 Bonus: Reverse Order Deletions & Comparison **14**

Rules Reference

Array used in every problem: arr $= \{5, 12, 8, 21, 3, 17, 9\}$, $n = 7$.

Creation Rules (C1–C6):

- **C1 (Root Creation):** If the tree is empty, place the first element at index 0 as the root.
- **C2 (Sequential Array-to-Tree Mapping):** Element at index i is placed in level-order left-to-right.
- **C3 (Left Child Formula):** For parent node at index i, left child is at index $2i + 1$.
- **C4 (Right Child Formula):** For parent node at index i, right child is at index $2i + 2$.
- **C5 (Parent Index Formula):** For child node at index $i > 0$, parent node is at index $\lfloor (i - 1)/2 \rfloor$.
- **C6 (Complete Tree Property):** An array representation with n contiguous elements forms a complete binary tree where every level except possibly the last is completely filled, and all nodes are as far left as possible.

Traversal Rules (T1–T6):

- **T1 (Base Case):** Visiting a NULL node returns immediately.
- **T2 (Preorder Sequence):** Process Node, recurse Left child, recurse Right child.
- **T3 (Inorder Sequence):** Recurse Left child, process Node, recurse Right child.
- **T4 (Postorder Sequence):** Recurse Left child, recurse Right child, process Node.
- **T5 (Level-Order Queue Rules):** Enqueue root. While queue is non-empty: Dequeue node v, output v, enqueue left child (if non-NULL), enqueue right child (if non-NULL).
- **T6 (Traversal Termination):** Return when all nodes in the tree/subtree have been visited.

Insertion Rules (I1–I6):

- **I1 (Empty Tree Insertion):** Place node at index 0 as root.
- **I2 (Level-Order Search for Free Spot):** Scan tree nodes in level order using a queue until finding the first node missing a child.
- **I3 (Attach as Left Child):** If parent has no left child, attach new node as left child (index $2i + 1$).
- **I4 (Attach as Right Child):** If parent has left child but no right child, attach new node as right child (index $2i + 2$).
- **I5 (Full Parent Skip):** If parent has both left and right children, skip parent and continue level-order scan.
- **I6 (Direct Array Index Calculation):** For a complete binary tree with N current elements, the new element goes directly to index $i = N$, parent at $\lfloor (N - 1)/2 \rfloor$; left child if N is odd, right child if N is even.

Deletion Rules (D1–D7):

- **D1 (Empty Tree Guard):** Deleting from an empty tree returns failure/NULL.
- **D2 (Target Not Found):** If target element is not present in the tree, tree remains unchanged.
- **D3 (Find Target and Deepest-Rightmost Node):** Locate target node T and the deepest, rightmost node L (index $N - 1$) via level-order scan.
- **D4 (Target Search Failure):** If target node T is not found during scan, stop and return.
- **D5 (Value Replacement):** Copy the value from deepest node L into target node T.
- **D6 (Remove Deepest Node):** Sever the link between L and its parent at index $\lfloor (N - 1)/2 \rfloor$.
- **D7 (Direct Single Node Removal):** If target T is itself the deepest node L (including a single-node tree or leaf L), directly delete L without value copying.

1 Problem 1: Creating the Tree

Given array arr $= \{5, 12, 8, 21, 3, 17, 9\}$ with $n = 7$.

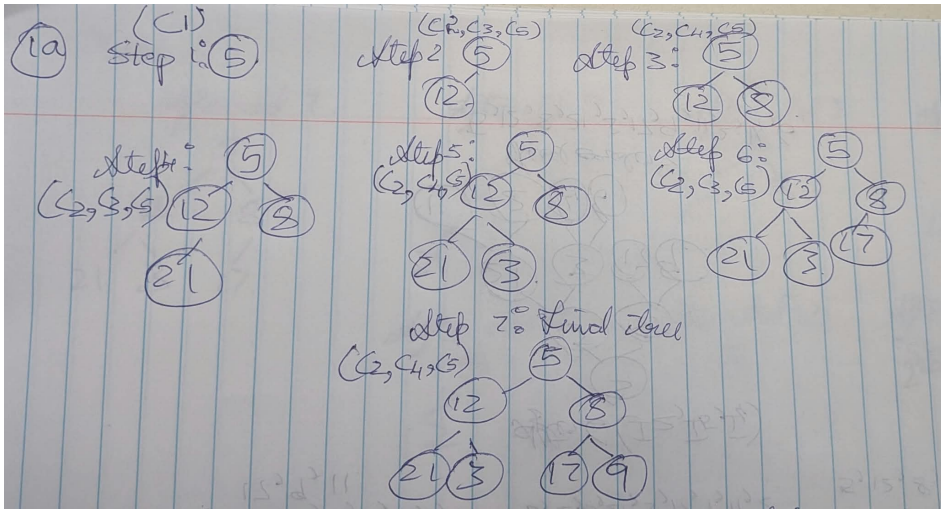
(a) Step-by-Step Tree Construction Table & Diagrams

Table 1: Step-by-Step Array Insertion into Complete Binary Tree

Step	Element	Index i	Parent Index	Parent Value	Left / Right	Rule(s)
1	5	0	N/A	N/A	Root	C1
2	12	1	0	5	Left	C2, C3, C5
3	8	2	0	5	Right	C2, C4, C5
4	21	3	1	12	Left	C2, C3, C5
5	3	4	1	12	Right	C2, C4, C5
6	17	5	2	8	Left	C2, C3, C5
7	9	6	2	8	Right	C2, C4, C5

Parent index check: step 2: $\lfloor (1-1)/2 \rfloor = 0$; step 3: $\lfloor 1/2 \rfloor = 0$; step 4: $\lfloor 2/2 \rfloor = 1$; step 5: $\lfloor 3/2 \rfloor = 1$; step 6: $\lfloor 4/2 \rfloor = 2$; step 7: $\lfloor 5/2 \rfloor = 2$. An odd index is a left child and an even index (> 0) is a right child.

Tree drawn after every step:



Final tree in level order: $5, 12, 8, 21, 3, 17, 9$.

(b) NULL Children Analysis

The leaves $21, 3, 17, 9$ (indices $3, 4, 5, 6$) each have **two NULL children**. The nodes $5, 12, 8$ (indices $0, 1, 2$) have no NULL children.

- **Rules C3 and C4:** The children of index i would be at $2i + 1$ and $2i + 2$. For the leaves ($i = 3, \ldots, 6$) these indices are 7 through 14, all $\geq n = 7$. No array element exists there, so those child positions are NULL.
- **Rule C6:** In a complete tree, levels 0 and 1 and 2 are completely filled with $1 + 2 + 4 = 7$ nodes, so NULL children can only occur below the last level, i.e. at the children of the bottom-level nodes. For $n = 7$ the bottom level is level 2, so the NULLs belong to the four leaves.

(c) Tree Structure Metrics

- **Root:** 5
- **Leaves:** $21, 3, 17, 9$
- **Internal Nodes:** $5, 12, 8$
- **Number of Edges:** $n - 1 = 7 - 1 = 6$
- **Height (H):** 2 (edges from root to deepest leaf)
- **Depth of 17:** 2 (edges from root: $5 \to 8 \to 17$)
- **Ancestors of 3:** $12, 5$
- **Descendants of 12:** $21, 3$
- **Subtree Rooted at 8:** Root 8, left child 17, right child 9.
- **Path from 5 to 9:** $5 \to 8 \to 9$; **Path Length:** 2 edges.

(d) Verification of Binary Tree Properties

- **Max nodes on Level 2 (2^L):** $2^2 = 4$; the tree has exactly 4 $(21, 3, 17, 9)$. **True**
- **Max nodes for Height 2 ($2^{H+1} - 1$):** $2^3 - 1 = 7$; the tree has exactly 7. **True**
- **Leaves = (nodes with two children) +1:** Nodes with two children = 3 $(5, 12, 8)$; leaves $= 3 + 1 = 4$ $(21, 3, 17, 9)$. **True**
- **Minimum height for $N = 7$:** $\lfloor \log_2 7 \rfloor = \lfloor 2.807 \rfloor = 2$, which equals the actual height. **True**
- **Classification:** The tree is **both Full and Complete**.
  - **Full:** Every node has either 0 children $(21, 3, 17, 9)$ or 2 children $(5, 12, 8)$.
  - **Complete (Rule C6):** Every level is completely filled $(1 + 2 + 4 = 7$ nodes) and the 7 elements occupy contiguous indices 0 to 6, so there are no gaps.

2 Problem 2: Traversing the Tree

Original Tree: Root = 5; Left = 12 (Left = 21, Right = 3); Right = 8 (Left = 17, Right = 9).

(a) Preorder Traversal (Node, Left, Right)

Sequence: $5, 12, 21, 3, 8, 17, 9$

- **5:** It is the root, and preorder processes a node *before* its subtrees (T2), so it is visited first.
- **12:** After 5, recurse into the left child of 5. 12 is processed on arrival, before its own children (T2).
- **21:** Recurse left from 12. 21 is processed on arrival (T2). Its left and right children are NULL, so both calls return at once (T1).
- **3:** The left subtree of 12 is finished, so recurse right from 12. 3 is processed on arrival (T2); its NULL children return (T1). The subtree of 12 is complete, so we return to 5.
- **8:** The left subtree of 5 is finished, so recurse right from 5. 8 is processed on arrival (T2).
- **17:** Recurse left from 8. 17 is processed on arrival (T2); NULL children return (T1).
- **9:** Recurse right from 8. 9 is processed on arrival (T2); NULL children return (T1).
- All 7 nodes have been visited, so the traversal terminates (T6).

(b) Inorder Traversal (Left, Node, Right)

Sequence: $21, 12, 3, 5, 17, 8, 9$

- **21:** Go left from $5 \to 12 \to 21$. The left child of 21 is NULL (T1), so 21 is the first node processed (T3). Its right child is NULL (T1); return to 12.
- **12:** Its left subtree is done, so 12 is processed (T3). Then recurse right.
- **3:** Left of 3 is NULL (T1), so 3 is processed (T3). Right of 3 is NULL (T1). The left subtree of 5 is done.
- **5:** Its left subtree is complete, so 5 is processed (T3). Then recurse right to 8.
- **17:** Go left from 8. Left of 17 is NULL (T1), so 17 is processed (T3); its right is NULL (T1).
- **8:** Its left subtree is done, so 8 is processed (T3). Then recurse right.
- **9:** Left of 9 is NULL (T1), so 9 is processed (T3); its right is NULL (T1).
- All nodes visited, the traversal terminates (T6).

(c) Postorder Traversal (Left, Right, Node)

Sequence: $21, 3, 12, 17, 9, 8, 5$

- **21:** Go left $5 \to 12 \to 21$. Both children of 21 are NULL (T1), so 21 is processed (T4).
- **3:** Back at 12, recurse right to 3. Both children NULL (T1), so 3 is processed (T4).
- **12:** Both subtrees of 12 are finished, so 12 is processed (T4).
- **17:** Back at 5, recurse right to 8, then left to 17. Both children NULL (T1), so 17 is processed (T4).
- **9:** Recurse right from 8 to 9. Both children NULL (T1), so 9 is processed (T4).
- **8:** Both subtrees of 8 are finished, so 8 is processed (T4).
- **5:** Both subtrees of 5 are finished, so the root 5 is processed last (T4).
- All nodes visited, the traversal terminates (T6).

(d) Level Order Traversal Queue Table (Rule T5)

Table 2: Level Order Traversal Queue Trace

#	Queue Before	Remove & Visit	Add Children (L, R)	Queue After	Output So Far
1	[5]	5	12, 8	[12, 8]	5
2	[12, 8]	12	21, 3	[8, 21, 3]	5, 12
3	[8, 21, 3]	8	17, 9	[21, 3, 17, 9]	5, 12, 8
4	[21, 3, 17, 9]	21	None	[3, 17, 9]	5, 12, 8, 21
5	[3, 17, 9]	3	None	[17, 9]	5, 12, 8, 21, 3
6	[17, 9]	17	None	[9]	5, 12, 8, 21, 3, 17
7	[9]	9	None	[ ]	5, 12, 8, 21, 3, 17, 9

(e) Analytical Questions

- **Why Level Order equals the original array:** Rule C2 places the array elements into the tree in level order, left to right: root at index 0, level 1 at indices $1, 2$, level 2 at indices $3, 4, 5, 6$. Level-order traversal (T5) visits nodes level by level from top to bottom and left to right, which is exactly index order 0 through $n - 1$. So the output is $5, 12, 8, 21, 3, 17, 9$, the original array.
- **Which traversal visits the root last: Postorder.** Rule T4 processes the left subtree and the right subtree completely before the node itself, so the root is processed after every other node.

(f) Searching Comparisons (Level Order)

- **Search for 3:** $5 \neq 3 \rightarrow 12 \neq 3 \rightarrow 8 \neq 3 \rightarrow 21 \neq 3 \rightarrow 3 = 3$. Found at index 4 after **5 comparisons**.
- **Search for 4:** Compare with $5, 12, 8, 21, 3, 17, 9$; none equals 4. Not found after **7 comparisons** (every node checked).

3 Problem 3: Adding Elements to the Tree

(a) Case I1: Insert into an Empty Tree

The tree is empty and 5 is inserted. It goes to index 0 and becomes the root. **Applied rule: I1 (Empty Tree Insertion).**

```
Before: (empty)        After:  5
```

(b) Sequential Insertions Starting from the Problem 1 Tree

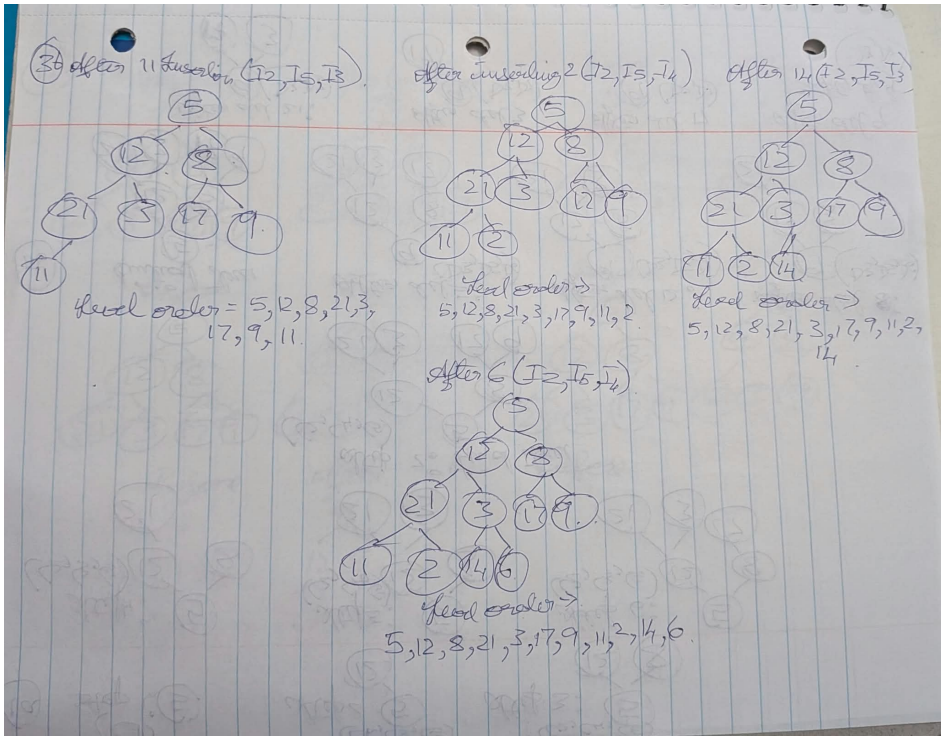
Insert 11. Level-order scan (I2) from the root:

- Node 5 (index 0): left = 12 present, right = 8 present → full, skip (I5).
- Node 12 (index 1): left = 21 present, right = 3 present → full, skip (I5).
- Node 8 (index 2): left = 17 present, right = 9 present → full, skip (I5).
- Node 21 (index 3): left child **missing** → attach 11 as the **left** child of 21 (I3). Index $2(3) + 1 = 7$.

Table 3: Insertion Trace Table for Problem 3

Insert	Nodes Checked (in order)	Rule at Each Node	Parent	Left / Right	Level Order After
11	5 (L, R present), 12 (L, R present), 8 (L, R present), 21 (no left child)	I5, I5, I5, I3	21	Left	5, 12, 8, 21, 3, 17, 9, 11
2	5 (full), 12 (full), 8 (full), 21 (left = 11 present, no right child)	I5, I5, I5, I4	21	Right	5, 12, 8, 21, 3, 17, 9, 11, 2
14	5 (full), 12 (full), 8 (full), 21 (full), 3 (no left child)	I5, I5, I5, I5, I3	3	Left	5, 12, 8, 21, 3, 17, 9, 11, 2, 14
6	5 (full), 12 (full), 8 (full), 21 (full), 3 (left = 14 present, no right child)	I5, I5, I5, I5, I4	3	Right	5, 12, 8, 21, 3, 17, 9, 11, 2, 14, 6

Tree drawn after every insertion:



(c) Why Element 2 Goes to the Right Side of 21

When 2 is inserted, the level-order scan (I2) skips 5, 12 and 8 (I5) and stops at 21 (index 3), the first node missing a child. This time 21 already has a left child (11, index 7), but its right position (index $2(3)+2=8$) is empty. **Rule I4** decides: if the parent has a left child but no right child, the new node attaches as the right child. When 11 was inserted, 21 had no left child, so **I3** placed it on the left. Same parent, different side, because the earlier insertion filled the left slot.

(d) Nodes Skipped for 14

Before 14 is inserted, nodes 5 (index 0), 12 (index 1), 8 (index 2) and 21 (index 3) all have both children (21 has 11 and 2). **Rule I5** skips each of these four full nodes. The scan stops at node 3 (index 4), which has no left child, so 14 is attached as its left child by **I3**.

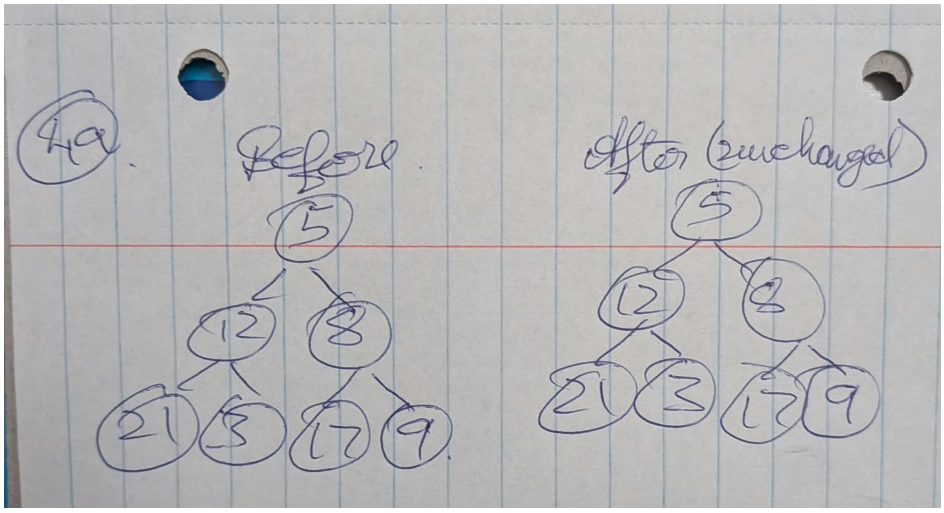
(e) Shortcut Prediction (Rule I6) vs. Full Scan for 6

- **Current number of elements:** $N = 10$ $(5, 12, 8, 21, 3, 17, 9, 11, 2, 14)$.
- **Target index:** $i = N = 10$.
- **Parent index:** $\lfloor (10-1)/2 \rfloor = \lfloor 4.5 \rfloor = 4$.
- **Parent value:** the element at index 4 is 3.
- **Side:** $N = 10$ is even, so by I6 the new node is the **right** child (index $2(4)+2=10$ by C4).
- **Prediction:** 6 goes to index 10, as the right child of 3.
- **Full scan check:** Nodes at indices 0, 1, 2, 3 $(5, 12, 8, 21)$ are all full, so they are skipped (I5). At index 4 (value 3) the left child 14 is present and the right child is missing, so I4 attaches 6 as the right child of 3. This matches the shortcut exactly.

4 Problem 4: Deleting Elements (Every Case)

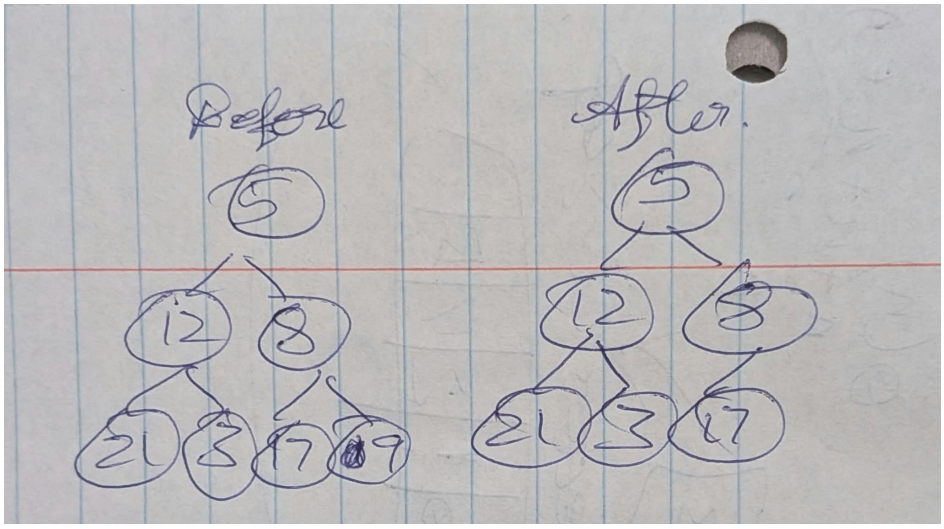
Each case starts fresh from the original tree T_0: level order $\{5, 12, 8, 21, 3, 17, 9\}$, $N = 7$. The deepest, rightmost node L is at index $N - 1 = 6$, value **9** (right child of 8).

(a) Case A: Delete 4 (Target Not in the Tree)



- **Rules:** D3 (scan for target T and deepest node $L = 9$), D4 (target not found, stop), D2 (tree remains unchanged).
- The level-order scan compares 4 with all 7 nodes and finds no match.
- **Result:** The tree is unchanged because there is nothing to delete. Level order: $5, 12, 8, 21, 3, 17, 9$.

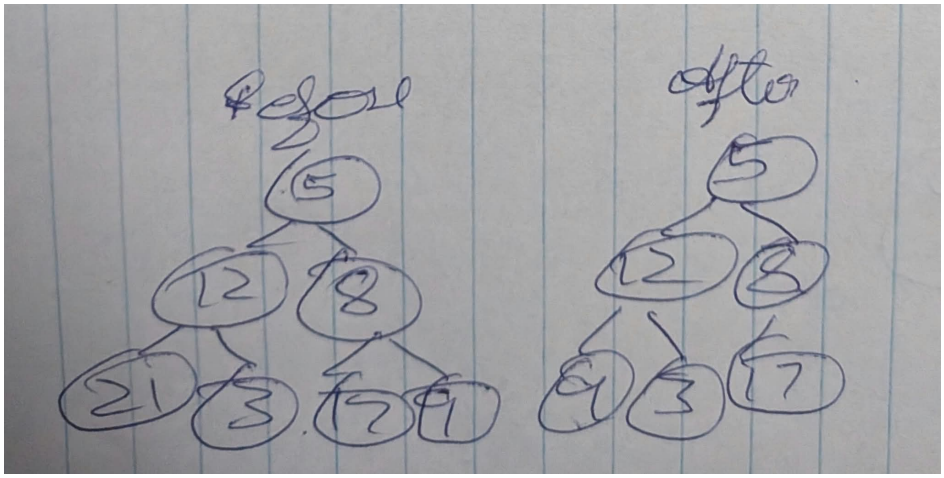
(b) Case B: Delete 9 (the Deepest Node)



- **Rules:** D3, D7.
- $T = 9$ is at index 6, which is also L.

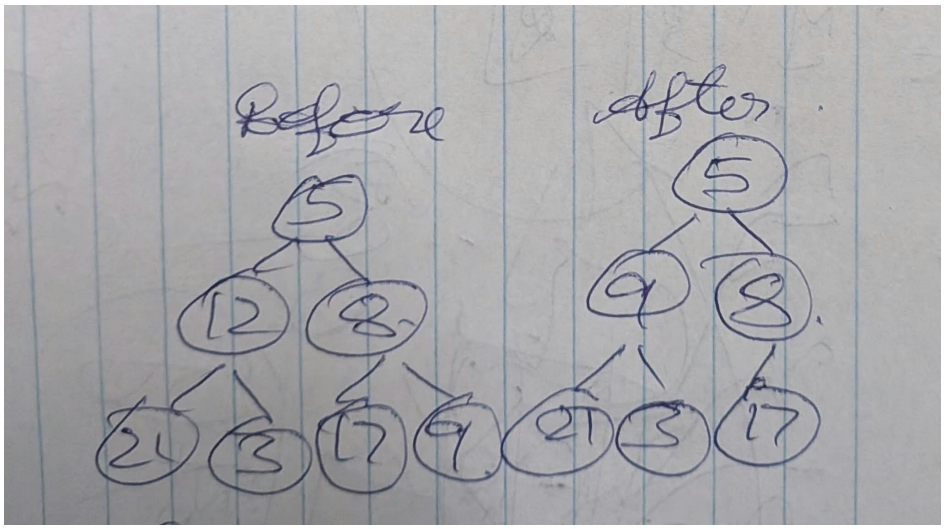
- **Why this is the easiest case:** Since $T = L$, no value has to be copied (no D5). The link from the parent 8 to its right child is simply set to NULL.
- **Level order after:** $5, 12, 8, 21, 3, 17$.

(c) Case C: Delete 21 (a Leaf That Is Not the Deepest Node)



- **Rules:** D3, D5, D6.
- $T = 21$ (index 3); $L = 9$ (index 6, child of 8).
- D5: copy 9 into index 3, replacing 21. D6: sever the old node L (index 6) from its parent at index $\lfloor (6-1)/2 \rfloor = 2$, which is the node 8.
- **Level order after:** $5, 12, 8, 9, 3, 17$.

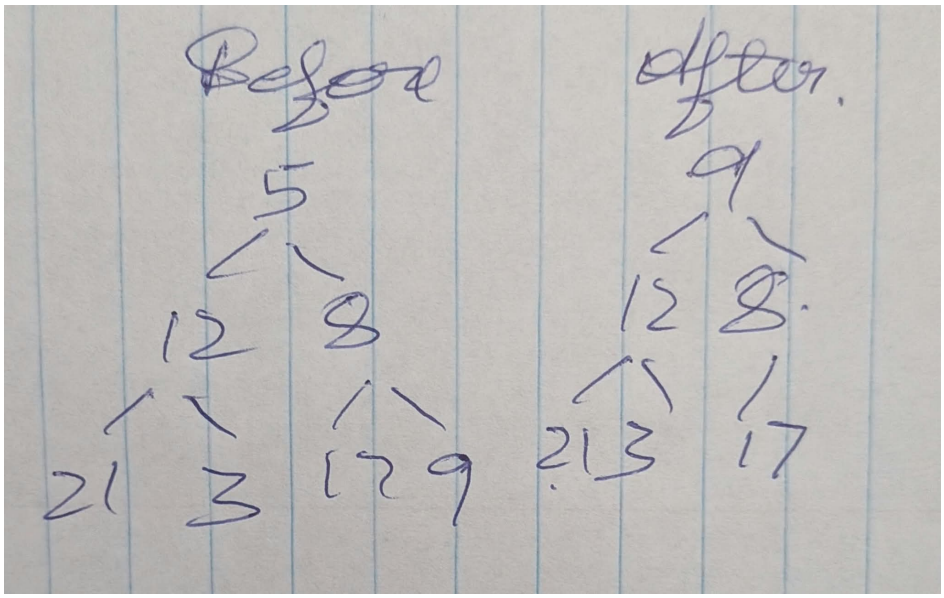
(d) Case D: Delete 12 (an Internal Node)



- **Rules:** D3, D5, D6.
- $T = 12$ (index 1); $L = 9$ (index 6).

- D5: copy 9 into index 1. D6: remove the node at index 6 (the right child of 8).
- **Children of the replacement node:** The node 9 now at index 1 keeps the positions that 12 had, so its **left child is 21** (index 3) and its **right child is 3** (index 4).
- **Level order after:** $5, 9, 8, 21, 3, 17$.

(e) Case E: Delete 5 (the Root)



- **Rules:** D3, D5, D6.
- $T = 5$ (index 0); $L = 9$ (index 6).
- D5: copy 9 into index 0. D6: remove the node at index 6.
- **New root: 9**.
- **Level order after:** $9, 12, 8, 21, 3, 17$.

(f) Case F: Single-Node Tree $\{9\}$

F1: delete 9 from $\{9\}$ – before

```
9
```

F1 – after

```
(empty tree)
```

F2: delete 4 from $\{9\}$ – before

```
9
```

F2 – after (unchanged)

```
9
```

- **F1 (delete 9): Rules D3, D7.** $T = 9$ is at index 0 and is also L (the only node). It is removed directly, with no copying, and the tree becomes **empty**.
- **F2 (delete 4 on a fresh $\{9\}$): Rules D3, D4, D2.** The scan compares 4 with the only node 9 and finds no match, so the tree stays $\{9\}$.

(g) Case G: Delete 5 from an Empty Tree

```
Before: (empty tree)        After: (empty tree)
```

- **Rule D1 (Empty Tree Guard):** $N = 0$, so there is nothing to search. The operation returns failure/NULL immediately and the tree remains empty.

(h) Theoretical Explanation

Why move the deepest node into the target's place instead of simply removing the target?
In the array representation, the tree stays **complete** (Rule C6) only if the $N-1$ remaining elements occupy contiguous indices 0 to $N-2$. Removing an interior node would leave a gap in the array (an empty slot in the middle) and would cut off the subtrees that hang below it. Replacing the target's value with the deepest, rightmost node L (D5) and then removing L (D6) only ever deletes the *last* index $N-1$. All other nodes keep their positions, no gap appears, and the child/parent index formulas ($2i+1$, $2i+2$, $\lfloor (i-1)/2 \rfloor$) remain valid.

(i) Summary Table for Problem 4

Table 4: Deletion Trace Table for Problem 4

Case	Delete	Target	Deepest	Rules Applied	Level Order After
A	4	Not found	9 (idx 6)	D3, D4, D2	5, 12, 8, 21, 3, 17, 9
B	9	9 (idx 6)	9 (idx 6)	D3, D7	5, 12, 8, 21, 3, 17
C	21	21 (idx 3)	9 (idx 6)	D3, D5, D6	5, 12, 8, 9, 3, 17
D	12	12 (idx 1)	9 (idx 6)	D3, D5, D6	5, 9, 8, 21, 3, 17
E	5	5 (idx 0)	9 (idx 6)	D3, D5, D6	9, 12, 8, 21, 3, 17
F	9 / 4	9 (idx 0) / not found	9 (idx 0)	D3, D7 (for 9) / D3, D4, D2 (for 4)	Empty / {9}
G	5	None (empty tree)	None	D1	Empty

5 Problem 5: Deleting All Elements Sequentially

Sequential deletions **without resetting** the tree: delete $5, 12, 8, 21, 3, 17, 9$ in that order, starting from T_0.

(a) Master Deletion Trace Table

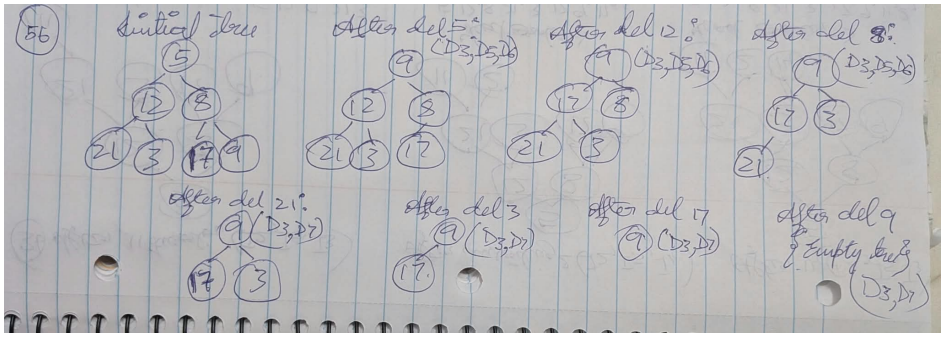
Table 5: Sequential Deletion Trace Table

#	Delete	Tree Before (Level Order)	Deepest	Case	Rules	Tree After (Level Order)
1	5	5, 12, 8, 21, 3, 17, 9	9	E	D3, D5, D6	9, 12, 8, 21, 3, 17
2	12	9, 12, 8, 21, 3, 17	17	D	D3, D5, D6	9, 17, 8, 21, 3
3	8	9, 17, 8, 21, 3	3	C	D3, D5, D6	9, 17, 3, 21
4	21	9, 17, 3, 21	21	B	D3, D7	9, 17, 3
5	3	9, 17, 3	3	B	D3, D7	9, 17
6	17	9, 17	17	B	D3, D7	9
7	9	9	9	F	D3, D7	Empty

Notes on the cases:

- #1: 5 is the root and $L = 9$, so this is Case E.
- #2: 12 has children 21 and 3 (internal node), and $L = 17$, so this is Case D.
- #3: In $9, 17, 8, 21, 3$, node 8 (index 2) is a leaf (its children would be at indices 5, 6, which do not exist) and $L = 3$ (index 4) is a different node. A leaf that is not the deepest node is Case C. 3 is copied into index 2.
- #4–#6: the target is the last index, so $T = L$ (Case B).
- #7: only one node remains, so this is Case F.

(b) Tree Drawn After Every Deletion



(c) Structural Analysis

- **Cases that appeared:** Case E (root deletion, step 1), Case D (internal node, step 2), Case C (leaf that is not the deepest node, step 3), Case B (target is the deepest node, steps 4–6) and Case F (single-node tree, step 7).
- **Was the tree still complete after every step? Yes.** Every deletion removes exactly the node at index $N - 1$: either directly when $T = L$ (D7), or by copying L into T (D5) and then severing L (D6). The $N - 1$ remaining elements therefore always occupy the contiguous indices 0 to $N - 2$ with no gaps, which is the complete-tree condition of Rule C6.

6 Bonus: Reverse Order Deletions & Comparison

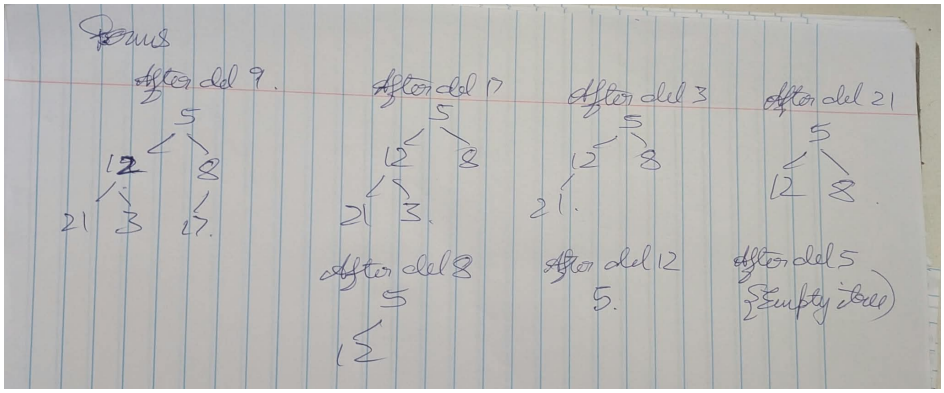
Delete in the reverse order $9, 17, 3, 21, 8, 12, 5$, starting from T_0.

(a) Reverse Deletion Trace Table

Table 6: Reverse Order Deletion Trace Table

#	Delete	Tree Before (Level Order)	Deepest	Case	Rules	Tree After (Level Order)
1	9	5, 12, 8, 21, 3, 17, 9	9	B	D3, D7	5, 12, 8, 21, 3, 17
2	17	5, 12, 8, 21, 3, 17	17	B	D3, D7	5, 12, 8, 21, 3
3	3	5, 12, 8, 21, 3	3	B	D3, D7	5, 12, 8, 21
4	21	5, 12, 8, 21	21	B	D3, D7	5, 12, 8
5	8	5, 12, 8	8	B	D3, D7	5, 12
6	12	5, 12	12	B	D3, D7	5
7	5	5	5	F	D3, D7	Empty

Tree drawn after every deletion:



(b) Analysis & Comparative Answers

- **Which case applies at every step? Case B** applies at steps 1–6 (the target is the deepest node), and **Case F** applies at step 7 (single-node tree).
- **Why is this order the simplest?** The values are deleted in exact reverse index order $(6, 5, 4, 3, 2, 1, 0)$. At each step the element to delete is therefore already the last node of the level order, so $T = L$ every time. Rule D7 removes it directly, with no searching for a replacement and no value copying.
- **Number of values copied (Rule D5) in each order:**
  - **Forward order (Problem 5): 3 copies** (step 1: 9 copied into the root; step 2: 17 copied into index 1; step 3: 3 copied into index 2).
  - **Reverse order (Bonus): 0 copies**. Every deletion used D7 only.